📊 DataSens E1 — Data Extraction & Transformation Pipeline

Status Production Ready | Python 3.10+ | License MIT | E1 v1.0.0 Complete

---

```
╔═══════════════════════════════════════════════════════════════════╗
║═══════╕ ═══════╕ ═══════╕ ═══════╕ ═══════╕═══════╕ ═══╕ ═══════╕
│       │ │     │ │       │ │     │ │       │       │   │ │       │
│       │ │     │ │       │ │     │ │       │       │   │ │       │
│       │ │     │ │       │ │     │ │       │       │   │ │       │
```

- `feedparser` — RSS extraction
- `requests`, `beautifulsoup4` — HTTP & web scraping
- `sqlalchemy` — Database ORM

2. Initialize Database

```
python scripts/setup_with_sql.py
```

Creates SQLite database with:

- Tables métier E1 (`source`, `raw_data`, `sync_log`, `topic`, `document_topic`, `model_output`, etc.)
- Lignes `source` initialisées depuis la configuration du script (alignées avec `sources_config.json`)
- Index pour les jointures fréquentes

3. Variables d'environnement (optionnel)

Copier `.env.example` vers **`.env`** à la racine du dépôt (non versionné). Exemples utiles pour E1 :

- **`DB_PATH`** — chemin du SQLite (défaut : `~/datasens_project/datasens.db`).
- **`OPENWEATHERMAP_API_KEY`** — si défini, la source `openweather_api` appelle **OpenWeatherMap** en priorité ; sinon repli **Open-Meteo** (sans clé).

Le pipeline charge `.env` au démarrage (`src/e1/pipeline.py`).

4. Run E1 Pipeline

```
python main.py
```

Inclus dans le même run (toutes les sources `active: true` de `sources_config.json`) : RSS, API, datasets, **scraping** (`trustpilot_reviews` en HTTP ; **`monavis_citoyen`** actif par défaut — **Botasaurus** en priorité puis navigateur Botasaurus puis `requests` ; `ifop_barometers` en HTTP uniquement si vous passez `active: true`, cadence plutôt annuelle). Les artefacts Botasaurus vont sous **`output/`** (gitignoré, ex. `_botasaurus_request.json`). Les sources `active: false` sont listées en fin d'extraction.

Output:

- Articles extraits puis chargés en base (avec déduplication)
- Table **`sync_log`** : une entrée par source et par run (`rows_synced` = nombre d'articles **retournés par l'extracteur** ce run-là)
- **Export RAW/SILVER/GOLD** inclus dans le pipeline

5. Export ou régénération (optionnel)

L'export est déjà effectué par `main.py`. Pour régénérer manuellement les exports :

```
python scripts/regenerate_exports.py
```

Produces:

- ⚫ `data/raw/sources_2025-12-16/` (JSON + CSV)
- ⚪ `data/silver/v_2025-12-16/` (Parquet)
- 🟡 `data/gold/date=2025-12-16/` (PySpark parquet)

🗂 Three-Zone Architecture

```
⚫ RAW ZONE (Native Formats - NO Processing)
  data/raw/sources_2025-12-16/
  ├─ raw_articles.json (137.4 KB) ← Direct from extractors
  └─ raw_articles.csv  (100.4 KB)  ← No transformations

⚪ SILVER ZONE (Cleaned & Standardized)
  data/silver/v_2025-12-16/
  └─ silver_articles.parquet (64.5 KB)
     • Deduplicated (fingerprint-based)
     • Quality scores (0-1 scale)
     • Topic tagging (8 topics, multiple per article)
     • Text cleaning indicators

🟡 GOLD ZONE (ML-Enriched, PySpark Ready)
  data/gold/date=2025-12-16/
  └─ articles.parquet (67.9 KB)
     • Sentiment analysis (176 neutral, 32 positive, 8 negative)
     • Confidence scores (0-1)
     • Processing metadata
     • Partitioned for PySpark
```

📚 Core Files

File	Purpose	Status
main.py	E1 pipeline orchestration (inclut export RAW/SILVER/GOLD)	✅
scripts/setup_with_sql.py	Database initialization	✅
scripts/regenerate_exports.py	Régénération manuelle RAW → SILVER → GOLD	✅
src/e1/core.py	Extracteurs et transformers E1	✅
sources_config.json	Liste des sources (`active`, URL, `acquisition_type`)	✅
requirements.txt	All dependencies	✅

> **ZZDB MongoDB** : pour activer la source `zzdb_synthetic`, installer `pymongo` (sinon la validation ZZDB affichera un warning).

Data Sources

La **liste à jour** (noms, URL, `acquisition_type`, `active`) est dans **`sources_config.json`**. Au moment de la rédaction du README, **16 sources** sont **`active: true`** (dont **`monavis_citoyen`**).

Sources actives (référence)

Source	**Type (`acquisition_type`)**	**Rôle**
rss_french_news	rss	Flux RSS
gdelt_events	bigdata	Fichiers GDELT (fondation, skip si déjà intégré)
reddit_france	api	API JSON Reddit
trustpilot_reviews	scraping	HTML (`requests` + BeautifulSoup)
openweather_api	api	Météo (OpenWeatherMap si clé `.env`, sinon Open-Meteo)
insee_indicators	api	INSEE (API + fallback site)
datagouv_datasets	dataset	Jeux data.gouv
kaggle_french_opinions	dataset	Kaggle (fondation, skip si déjà intégré)
google_news_rss	rss	RSS
yahoo_finance	rss	RSS
monavis_citoyen	**scraping**	**monaviscitoyen.fr** — Botasaurus (requête → navigateur) puis `requests` ; parse HTML dans `ScrapingExtractor` (`src/e1/core.py`)
agora_consultations	api	API Agora (JSON)
GDELT_Last15_English	bigdata	Échantillon GDELT
GDELT_Master_List	bigdata	Liste GDELT
zzdb_csv	csv	CSV ZZDB

Les **totaux d'articles en base par source** changent chaque jour : voir la section **Traçabilité des extractions** plus bas (`db_state_report`, SQLite, exports).

Classification:

- **Fondation** : intégration contrôlée (ex. `kaggle_french_opinions`, `gdelt_events`, `zzdb_csv`) — le pipeline peut **SKIP** si déjà marquée intégrée.
- **Dynamiques** : re-collecte à chaque run ; beaucoup d'articles extraits peuvent être **dédupliqués** (même empreinte titre+contenu) et ne pas augmenter `raw_data`.

Sources inactives (référence)

Source	Type	Remarque
Kaggle_StopWords_28Lang, Kaggle_StopWords, Kaggle_FrenchFinNews, Kaggle_SentimentLexicons, Kaggle_InsuranceReviews, Kaggle_FrenchTweets	dataset	`active: false` par défaut
ifop_barometers	scraping	`active: false` par défaut (cadence plutôt annuelle)
zzdb_synthetic	mongodb	Nécessite Mongo / config ZZDB

📌 Traçabilité des extractions (par source)

Objectif : suivre **ce qui a été extrait**, **ce qui est stocké depuis l'origine**, et **l'historique par run** — pour **toutes** les sources (scraping MonAvis, API, RSS, etc.).

1. Stock total par source (depuis le début)

Dans la base SQLite (`DB_PATH`, souvent `~/datasens_project/datasens.db`), table `raw_data` jointe à `source` : chaque ligne est un article **effectivement enregistré** (après déduplication).

Exemples Windows PowerShell / cmd (remplacer par le `DB_PATH` réel de ton `.env` si différent) :

```
sqlite3 "%USERPROFILE%\datasens_project\datasens.db" "SELECT s.name, COUNT(*) FROM raw_data r JOIN source s ON s.source_id = r.source_id GROUP BY s.name ORDER BY COUNT(*) DESC;"
```

Pour **MonAvis** uniquement :

```
sqlite3 "%USERPROFILE%\datasens_project\datasens.db" "SELECT COUNT(*) FROM raw_data r JOIN source s ON s.source_id = r.source_id WHERE s.name = 'monavis_citoyen';"
```

Sous Linux ou macOS, utiliser le chemin absolu vers le même fichier (souvent `~/datasens_project/datasens.db`).

2. Historique run par run (nombre extrait, pas seulement « nouveaux »)

Table `sync_log` : à chaque passage du pipeline sur une source, une ligne avec `rows_synced` = nombre d'articles **renvoyés par l'extracteur** pour ce run (avant dédup), `sync_date`, `status`.

Exemple — derniers enregistrements pour MonAvis :

```
sqlite3 "%USERPROFILE%\datasens_project\datasens.db" "SELECT sl.sync_date,
sl.rows_synced, sl.status FROM sync_log sl JOIN source s ON s.source_id =
sl.source_id WHERE s.name = 'monavis_citoyen' ORDER BY sl.sync_log_id DESC LIMIT
20;"
```

3. Fichiers « instantané » du jour (toutes sources dans un même fichier)

Après chaque `main.py`, le pipeline écrit sous **`data/raw/sources_YYYY-MM-DD/`** :

- **`raw_articles.json`** et **`raw_articles.csv`** — **toutes** les sources mélangées ; chaque enregistrement comporte une clé **`source`** (nom de la source, ex. `monavis_citoyen`). Tu peux filtrer dans un éditeur, Excel, ou en JSON avec `jq` pour ne garder que MonAvis.

4. Rapports versionnés (synthèse + `sync_log`)

```
python scripts/db_state_report.py
```

Génère **`reports/db_state_*.{md,json}`** avec :

- totaux **`raw_data`** ;
- **comptes par source** ;
- **historique récent** **`sync_log`** (lignes synchronisées / statut par source).

5. Exports globaux et dashboard

- **`exports/raw.csv`** (et SILVER/GOLD) : colonne source pour filtrer **MonAvis** ou une autre source.
- **`python scripts/show_dashboard.py`** : vue agrégée (articles par source, « nouveaux aujourd'hui », etc.).

6. Scraping Botasaurus (debug ponctuel)

Les traces **techniques** Botasaurus (requête brute) sont sous **`output/`** (gitignoré). Ce n'est **pas** le comptage métier : pour les volumes et l'historique, privilégier **`sync_log`**, **`raw_data`** et **`db_state_report`**.

📊 Dashboard de Visualisation

✅ Automatique et Dynamique

Le système inclut **3 outils de visualisation** qui se mettent à jour automatiquement :

1. **Rapport de Collecte** : S'affiche automatiquement après chaque `python main.py`
2. **Dashboard Global** : Vue d'ensemble complète avec `python show_dashboard.py`
3. **Visualiseur CSV** : Explorer les fichiers exports/ avec `python view_exports.py`

📋 Rapport de Collecte (Session Actuelle)

Le rapport s'affiche **automatiquement** après chaque collecte et montre :

- Articles collectés, taggés, analysés dans cette session
- Détail par source des nouveaux articles
- Distribution topics et sentiment des nouveaux articles

📊 Dashboard Global

```
# Afficher le dashboard complet (toujours à jour)
python scripts/show_dashboard.py
```

Affiche :

- **Résumé global** : Total articles, uniques, nouveaux aujourd'hui, enrichis
- **Nouveaux articles** : Détail par source des articles collectés aujourd'hui
- **Enrichissement Topics** : Articles taggés, topics utilisés, confiance moyenne
- **Enrichissement Sentiment** : Distribution positif/neutre/négatif
- **Articles par source** : Statistiques détaillées par source
- **Évaluation IA** : Status du dataset pour l'entraînement IA

👀 Visualiser les CSV dans exports/

```
# Script interactif pour explorer les CSV
python scripts/view_exports.py
```

Les fichiers CSV sont aussi directement accessibles dans `exports/` :

- `raw.csv` : Données brutes (DB + fichiers locaux)
- `silver.csv` : Données nettoyées avec topics
- `gold.csv` : Données complètes avec topics + sentiment

Vous pouvez les ouvrir directement dans Excel, Notepad, ou les importer dans Power BI.

🔄 Enrichir rétroactivement tous les articles

```
# Enrichir tous les articles existants (topics + sentiment)
python scripts/enrich_all_articles.py
```

📖 **Guide complet** : Voir `docs/DASHBOARD_GUIDE.md` pour plus de détails

📊 Data Quality Verification

Pipeline Summary ✓

- **Total Articles Extracted**: 81
- **Articles Cleaned**: 81
- **Articles Loaded to DB**: 10 (new today)

- **Total in DB**: 1017 (consolidated across runs)
- **Deduplication Rate**: 87.7%
- **Quality Score**: 100%

Export Outputs ✓

File	Records	Size	Format
raw.csv	1017	0.73 MB	CSV (raw data + Kaggle)
silver.csv	1017	0.17 MB	CSV (classified by DOCUMENT_TOPIC)
gold.csv	9	1 KB	CSV (aggregated by source)
gold.parquet	9	10 KB	Parquet (for Power BI)

Transformations Applied ✓

Transformation	Coverage	Status
Deduplication	71/81	✓
Quality Scoring	1017/1017	✓
Topic Classification	1017/1017 (8 topics)	✓
Sentiment Analysis	9 sources (MODEL_OUTPUT)	✓
Partitioned Raw Data	sources_2025-12-17	✓

🏗️ Database Schema

6 Core Tables (E1)

1. **source** — une ligne par source configurée (alignée sur `sources_config.json` / setup)
2. **raw_data** — articles stockés (après dédup), toutes sources
3. **sync_log** — historique des runs par source (`rows_synced`, `sync_date`, `status`)
4. **topic** — 8 predefined topics
5. **document_topic** — 207 article-topic mappings
6. **model_output** — 648 ML predictions

Note: `sqlite_sequence` est une table système SQLite (gérée automatiquement) qui stocke les compteurs AUTOINCREMENT. Elle n'est pas une table métier.

📈 Statistics

Metric	Value
Total Articles	216
Active Sources	10/10

Metric	Value
Topics Created	8
Duplicates Removed	0
Data Corruption	0
Quality Score Range	0.01 - 0.99
Model Outputs	648
Transformations	100% complete

✅ Production Readiness

✅ Code Quality

- Ruff linting: 100% pass
- Type hints throughout
- OOP architecture (SOLID principles)
- No hardcoded values

✅ Data Quality

- Zero duplicates
- Zero corruption
- 100% article coverage
- Proper deduplication

✅ Documentation

- README (this file)
- AGILE_ROADMAP.md (43 user stories)
- SCHEMA_DESIGN.md (database design)
- CHANGELOG.md (version history)
- **Conformité** : Audits E1–E5, RGPD, OWASP, monitoring, incidents (tout bâché)
- **docs/** : 60+ documents (architecture, flux, audits, procédures)

✅ Dependencies

- All listed in requirements.txt
- Pinned versions
- pandas, pyarrow, fastparquet installed

📝 File Structure

```
PROJET_DATASENS/
├── main.py                          # E1 orchestration (inclut export
RAW/SILVER/GOLD)
```

```
├── scripts/setup_with_sql.py        # Database setup
├── requirements.txt                 # Dependencies
├── sources_config.json              # Sources config
├── README.md                        # This file
├── pytest.ini                       # Configuration pytest
├── src/
│   ├── __init__.py
│   ├── e1/                          # E1 ISOLÉ (package privé)
│   │   ├── __init__.py
│   │   ├── core.py                  # Extracteurs et transformers
│   │   ├── repository.py            # Repository pattern
│   │   ├── tagger.py                # Topic tagger
│   │   ├── analyzer.py             # Sentiment analyzer
│   │   ├── aggregator.py            # Data aggregator
│   │   ├── exporter.py             # Gold exporter
│   │   └── pipeline.py             # E1Pipeline isolé
│   ├── e2/                          # E2 (FastAPI + RBAC) - PRÊT
│   │   └── __init__.py
│   ├── e3/                          # E3 (PySpark + ML) - PRÊT
│   │   └── __init__.py
│   ├── shared/                      # INTERFACES (contrats E1 ↔ E2/E3)
│   │   ├── __init__.py
│   │   └── interfaces.py           # E1DataReader (lecture seule)
│   ├── dashboard.py                 # Dashboard utilitaires
│   ├── collection_report.py         # Rapport de collecte
│   └── metrics.py                   # Prometheus metrics
├── tests/
│   ├── test_e1_isolation.py         # Tests non-régression E1
│   └── README_E1_ISOLATION.md       # Guide tests
├── docs/
│   ├── PLAN_ACTION_E1_E2_E3.md      # Plan d'action détaillé
│   ├── E1_ISOLATION_STRATEGY.md    # Stratégie isolation
│   ├── E1_ISOLATION_COMPLETE.md    # Récapitulatif Phase 0
│   └── ROADMAP_EVOLUTION.md         # Roadmap E1 → E2 → E3
└── data/
    ├── raw/
    │   └── sources_2025-12-20/
    │       ├── raw_articles.json
    │       └── raw_articles.csv
    ├── silver/
    │   └── v_2025-12-20/
    │       └── silver_articles.parquet
    └── gold/
        └── date=2025-12-20/
            └── articles.parquet
```

🎓 Key Concepts

Three-Zone Architecture

- **RAW**: Unprocessed data directly from sources (JSON/CSV)

- **SILVER**: Cleaned, standardized, deduplicated (Parquet)
- **GOLD**: ML-enriched, production-ready (PySpark Parquet)

Immutable Data Pipeline

1. No modifications to raw data (source of truth)
2. All transformations tracked
3. Lineage clearly documented
4. Easy to reprocess if needed

PySpark Ready

- GOLD zone uses `date=2025-12-16/` partitioning
- Compatible with PySpark's partitioned dataset format
- Can be read with: `spark.read.parquet("data/gold/")`

🔒 E1 Isolation (Phase 0)

Package `src/e1/` isolé. E2/E3 lisent via `E1DataReader` uniquement — pas de touche au code E1. Tests non-régression en place. Détails : `docs/E1_ISOLATION_COMPLETE.md`.

📋 Doc & conformité

Grilles E1–E5 : `AUDIT_E1_COMPETENCES.md` … `AUDIT_E5_COMPETENCES.md`, E4 = écarts + plan. E1/E2/E3/E5 validés.

RGPD / sécu : Registre traitements, procédure tri DP, OWASP Top 10 (dans README_E2_API).

Monitoring / incidents : Métriques, seuils, alertes, Prometheus/Grafana, Uptime Kuma, accessibilité. Procédure incidents prête. Guide captures (ReDoc, Prometheus, Grafana) : `docs/E1_CAPTURES_MONITORING.md`. Uptime Kuma est lancé via Docker (`docker-compose` / `start_uptime_kuma.bat`), pas via `pip`. **Métriques E1** : service `datasens-e1-metrics` (Docker) ou `python scripts/run_e1_metrics.py` en local.

Plan de lancement : `PLANCHE_LANCEMENT.md` — ordre démarrage MLflow, Docker, API, monitoring.

Index complet : `docs/README.md`.

🚀 Phases du Projet

Phase 0 : E1 isolé, `E1DataReader`, tests non-régression.

Phase 2 : FastAPI + RBAC, JWT, audit trail. Tests API OK.

Phase 3 : PySpark (singleton local), GoldParquetReader, 4 endpoints analytics. ~88k lignes Parquet GOLD.

Outils PySpark :

```
# Shell interactif PySpark
python scripts/pyspark_shell.py

# Tests rapides locaux
python scripts/test_spark_simple.py

# Tests complets
pytest tests/test_spark_integration.py -v
```

Endpoints : `sentiment/distribution`, `source/aggregation`, `statistics`, `available-dates`.

Phase 4 : FlauBERT/CamemBERT fine-tuning, **MLflow** (versioning automatique dans `finetune_sentiment.py`).
Phase 5 : Dashboard Streamlit.
Phase 6 : Mistral insights.

Plan détaillé : `docs/PLAN_ACTION_E1_E2_E3.md`

📄 License

MIT License — See LICENSE.md

🤝 Contributing

Contributions welcome! See CONTRIBUTING.md

📊 Parquet GOLD vs Base de Données

Différence entre Parquet et Dashboard E1

Le **dashboard E1** affiche le **total des articles dans la base de données SQLite** (`datasens.db`), qui contient **tous les articles collectés depuis le début** (43,022 articles au 20/12/2025).

Les **fichiers Parquet GOLD** sont **exportés par date** lors de chaque exécution du pipeline E1. Chaque fichier Parquet contient uniquement les articles **exportés pour cette date spécifique**.

Fichiers Parquet disponibles :

- `data/gold/date=2025-12-16/articles.parquet` : 216 lignes
- `data/gold/date=2025-12-18/articles.parquet` : 2,094 lignes
- `data/gold/date=2025-12-19/articles.parquet` : 42,466 lignes
- `data/gold/date=2025-12-20/articles.parquet` : 43,131 lignes

Total Parquet : 87,907 lignes (certains articles peuvent être dans plusieurs fichiers si exportés plusieurs fois)

Manipuler les Parquet avec PySpark

Utilisez le script interactif pour manipuler vos fichiers Parquet :

```
# Windows
scripts\manage_parquet.bat

# Linux/Mac
bash scripts/manage_parquet.sh
```

Fonctionnalités disponibles :

- ✅ Lire et afficher les données Parquet
- ✅ Filtrer les données (conditions SQL)
- ✅ Modifier les valeurs
- ✅ Ajouter des colonnes
- ✅ Supprimer des lignes
- ✅ Sauvegarder en nouveaux fichiers Parquet
- ✅ Appliquer des traitements (agrégations, statistiques)

Exemple d'utilisation :

1. Lancer le script : `python scripts/manage_parquet.py`
2. Choisir option `2` : Lire Parquet (toutes dates)
3. Choisir option `5` : Filtrer DataFrame (ex: `sentiment = 'positif'`)
4. Choisir option `9` : Sauvegarder en nouveau fichier Parquet

---

E2 Fine-tuning rapide (CPU ecole)

Un mode d'execution court est disponible pour produire une preuve C7/C8 exploitable sur machine CPU limitee.

Boucle complete recommandee

```
scripts\run_training_loop_e2.bat
```

Ce mode quick (par defaut) execute:

- regeneration des jeux `train/val/test` depuis GoldAI
- normalisation des labels sentiment avant split
- fine-tuning CamemBERT en volume borne
- benchmark final multi-modeles avec rapport dans `docs/e2/`

Mode complet (plus long)

```
scripts\run_training_loop_e2.bat --full
```

Le mode `--full` enleve les bornes d'echantillonnage et relance un entrainement complet (duree significativement plus elevee en CPU).

Artefacts produits

- `docs/e2/AI_BENCHMARK.md`
- `docs/e2/AI_BENCHMARK_RESULTS.json`
- `docs/e2/AI_REQUIREMENTS.md`
- `models/camembert-sentiment-finetuned/`

MLflow — Versioning des modèles

Chaque run de `finetune_sentiment.py` enregistre automatiquement dans MLflow (local `mlruns/`) :

- **Params** : model, epochs, mode, train/val samples
- **Metrics** : eval_accuracy, eval_f1_macro, eval_loss, train_runtime_seconds
- **Artifact** : config.json du modèle

```
# Consulter les runs après entraînement
mlflow ui
# → http://localhost:5000
```

Journal de session (2026-03-09)

Stabilisation technique E2

- Endpoint drift rendu resilient: `GET /api/v1/analytics/drift-metrics` bascule automatiquement sur un calcul `pandas` si Spark/Java n'est pas disponible.
- Lecture RAW durcie: un `raw_articles.csv` vide ne provoque plus de 500 (schema vide compatible API renvoye).
- Documentation API maintenue accessible via:
  - `http://localhost:8001/redoc`
  - `http://localhost:8001/openapi.json`

Industrialisation E3

- Quality gate E3 ajoute:
  - `tests/test_e3_quality_gate.py`
  - `scripts/run_e3_quality_gate.py`
  - `.github/workflows/e3-quality-gate.yml`
- Commande locale:

```
python scripts/run_e3_quality_gate.py
```

Consolidation documentation de soutenance

- E2 structure dans `docs/e2/` avec annexes de preuves/captures/demo.
- E3 structure dans `docs/e3/` avec dossier principal et annexes prêtes soutenance.

Last Updated: March 9, 2026
Version: 1.5.3
Status: Production Ready
E1/E2/E3 boucles. GoldAI merge operationnel. MLflow versioning. E1-metrics en continu (Docker). Quality gates E2/E3. Docs E5 complètes. **Plan complet** : `PLANCHE_LANCEMENT.md`.